

MATERIAL DE ESTUDIO

ALGORITMOS Y ESTRUCTURAS DE DATOS

*Ingeniería en Sistemas de Información
UTN Facultad Regional Resistencia*



```
Algoritmo (Nombre_algoritmo)
  Declaracion_de_variables
  INICIO
    INSTRUCCIÓN_1
    INSTRUCCIÓN_2
    INSTRUCCIÓN_3
    . . .
    INSTRUCCIÓN_N
  FIN_INICIO
Fin (Nombre_algoritmo)
```

CONCEPTOS Y PROBLEMAS

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Tabla de contenido

OBJETIVOS	3
CONTENIDOS	3
Conceptos Iniciales.....	4
Qué es un Algoritmo	4
Proceso, Acción y Estado.....	7
METODOLOGÍAS PARA LA RESOLUCIÓN DE ALGORITMOS.....	10
PSEUDOCÓDIGO.....	11
ELEMENTOS BÁSICOS DE LOS ALGORITMOS.....	12
Tipos de Datos.....	12
Estructuras de Datos simples	13
Variables.....	13
Constantes.....	14
Operadores y Operandos	16
Operadores.....	16
Prioridad de los Operadores	19
Prueba de Escritorio	19
¿Por qué debo usar la prueba de escritorio?	20
Pasos para elaborar una Prueba de Escritorio	20

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

OBJETIVOS

- a. Identificar las acciones involucradas en un problema.
- b. Reconocer los estados de los objetos antes y después de la acción realizada.
- c. Descomponer una acción en términos de otras más elementales.
- d. Utilizar un léxico mínimo común.
- e. Comprender el significado del término Algoritmo

CONTENIDOS

Estrategias de resolución de problemas: algoritmos, descripción narrada, pseudocódigos, diagramas, tablas de decisión. Concepto de acción, estado, procesador y proceso. Concepto de máquina, algoritmo y programa. Formas de representar los procesos. Esquemas. Concepto de dato y tipología de datos. Tipos elementales de datos. La operación asignación. Especificación de la acción a realizar en función de los estados. Concepto de estado intermedio. Descomposición de una acción en términos de otras más elementales. Composición secuencial de acciones. Elementos básicos de la notación algorítmica. Expresiones: tipos, operadores. precedencia de los operadores. Evaluación de las expresiones. Errores: tipos de errores. El concepto de prueba del algoritmo.

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

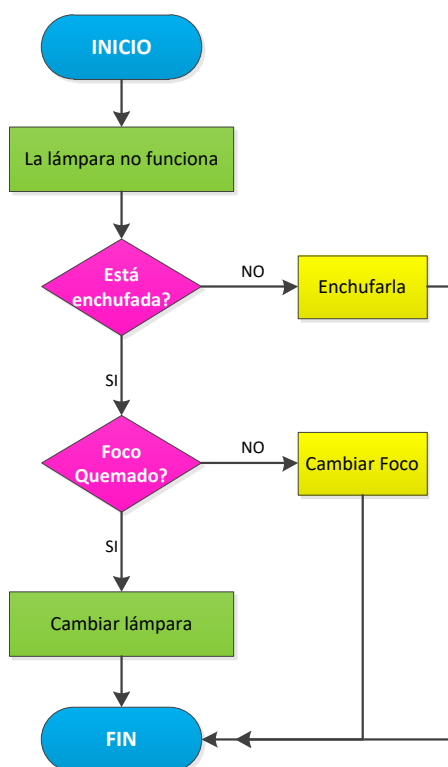
Conceptos Iniciales

Qué es un Algoritmo

¿Qué es un algoritmo? Bueno....no hay una sino varias definiciones que dan respuesta a esta pregunta, podríamos decir que un algoritmo:

- ✓ Es una fórmula para resolver un problema.
- ✓ Es un conjunto de acciones o secuencia de operaciones que ejecutadas en un determinado orden resuelven un problema.

Formalmente,



*Un **algoritmo** es una secuencia finita de instrucciones, reglas o pasos que describen de modo preciso las operaciones que una computadora debe realizar para ejecutar una tarea determinada en un tiempo finito.*

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

En este sentido entonces, para crear un algoritmo es necesario:

- | |
|--|
| 1. Comenzar identificando los resultados esperados , porque así quedan claros los objetivos a cumplir. Entender el problema y la necesidad de solución es imprescindible para avanzar. |
| 2. Luego, individualizar los datos con que se cuenta y determinar si con estos datos es suficiente para llegar a los resultados esperados. Es decir, definir los datos de entrada con los que se va a trabajar para lograr el resultado. |
| 3. Finalmente si los datos son completos y los objetivos claros, se intentan plantear los procesos necesarios para pasar de los datos de entrada a los datos de salida. |

En la práctica, un algoritmo es un método para resolver problemas mediante los pasos o etapas siguientes:

1. **Diseño del algoritmo** que describe la secuencia ordenada de pasos —sin ambigüedades— conducentes a la solución de un problema dado (Análisis del problema y desarrollo del algoritmo utilizando alguna técnica por ejemplo pseudocódigo).
2. **Expresar el algoritmo** como un programa en un lenguaje de programación adecuado. (Fase de codificación).
3. **Ejecución y validación** del programa por la computadora.

Para llegar a la realización de un programa es necesario el diseño previo de un algoritmo indicando cómo hace el algoritmo la tarea solicitada, y eso se traduce en la construcción de un algoritmo. El resultado final del diseño es una solución que debe ser fácil de traducir a estructuras de datos y estructuras de control de un lenguaje de programación específico.

Teniendo en cuenta el concepto de SISTEMA DE INFORMACIÓN, dado un problema, para plantear un algoritmo que permita resolverlo, es conveniente entender correctamente la situación problemática y su contexto, tratando de deducir del mismo los elementos ya indicados (entradas, procesos y salida).

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Características de los algoritmos

Las características fundamentales que debe cumplir todo algoritmo son:

- Un algoritmo debe ser **preciso** e indicar el orden de realización de cada paso.
- Un algoritmo debe estar bien **definido**. Si se sigue un algoritmo dos veces, se debe obtener el mismo resultado cada vez.
- Un algoritmo debe ser **finito**. Si se sigue un algoritmo, se debe terminar en algún momento; o sea, debe tener un número finito de pasos.
- Un algoritmo debe ser **correcto**: el resultado del algoritmo debe ser el resultado esperado.
- Un algoritmo es **independiente** tanto del lenguaje de programación en el que se expresa como de la computadora que lo ejecuta.

La definición de un algoritmo debe describir tres partes: *Entrada, Proceso y Salida*.

Ejemplo 1

Desarrolle un algoritmo que determine si un número es par o impar.

Solución

Primero vamos a indicar cuáles serían las partes necesarias:

- *Entrada*: Solicitar al usuario que ingrese un número X
- *Proceso*: Dividir por dos el número X.
- *Salida*: Si el resto de la división es 0, entonces informar el número X es par. Sino informar que es impar.

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Proceso, Acción y Estado

¿Qué es un proceso?

Es la unidad más pequeña de trabajo, individualmente planificable por un Sistema Operativo¹. Formalmente un proceso es **"Una unidad de actividad que se caracteriza por la ejecución de una secuencia de instrucciones, un estado actual, y un conjunto de recursos de los sistemas asociados"**

¿Qué es una acción?

Como habrás podido observar en el ejemplo 1 la descripción de cada uno de los pasos **comienza con un verbo (Solicitar, Ingresar, Dividir, ...)** y como todos sabemos, un verbo implica **acción**.

*Una **acción** es un acontecimiento producido por un **actor** que tiene un tiempo finito (período), produce un **resultado definido y preciso** y produce un cambio de **estado**.*



Situándonos en el ejemplo 1, entonces "Solicitar al usuario que ingrese un número X" ¿es una acción?

Claro que sí!! Posee un actor (pueden ser varios, dependiendo de la situación), dura un periodo de tiempo finito y produce un resultado definido.

¹ Un Sistema Operativo es el software encargado de ejercer el control y coordinar el uso del hardware entre diferentes programas de aplicación y los diferentes usuarios. Es un *administrador* de los recursos de hardware del sistema. Por ejemplo Windows 10, Linux, Android, etc.

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Te preguntarás: ¿Cómo puede ser que “construir una casa” es una acción y a su vez está formado por acciones?



Pues para responderte, las acciones pueden ser de dos tipos:

SIMPLES

son las que pueden ser realizadas directamente

COMPLEJAS

son las que no se pueden realizar directamente sino a través de una descomposición en acciones simples (de menor complejidad).

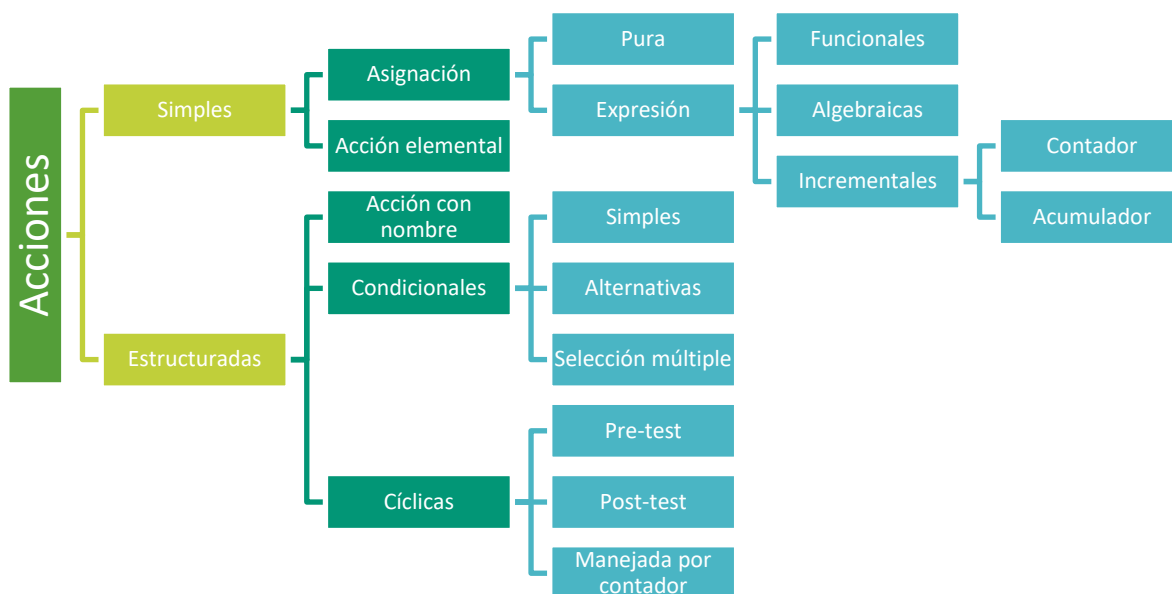


*La determinación de lo que es **complejo** y **simple** varía entre una persona y otra, y depende de sus capacidades y del enfoque que le quiera dar al problema*

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Clasificación de acciones



Más adelante durante el cursado de la materia vamos a ir trabajando con estas acciones.

Pero es MUY IMPORTANTE que puedas comprender cuándo usar cada tipo de acción

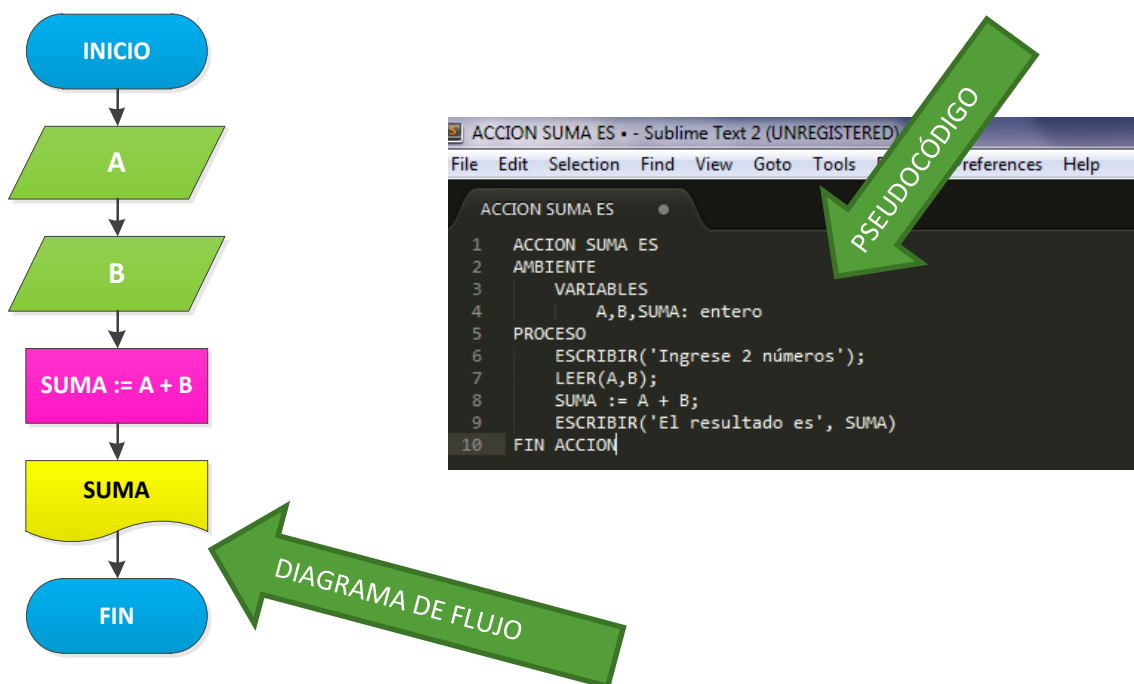
ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

METODOLOGÍAS PARA LA RESOLUCIÓN DE ALGORITMOS

Como se especificó anteriormente, un algoritmo es independiente del lenguaje de programación que se utilice. Es por esto, que existen distintas **técnicas de representación** de un algoritmo que permiten esta diferenciación con el lenguaje de programación elegido.

De esta forma el algoritmo puede ser representado en cualquier lenguaje. Existen diversas herramientas para representar gráficamente un algoritmo. En este curso presentaremos dos:



ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

PSEUDOCÓDIGO

El **pseudocódigo** nació como un lenguaje similar al inglés y era un medio para representar las estructuras de control de **programación estructurada**.

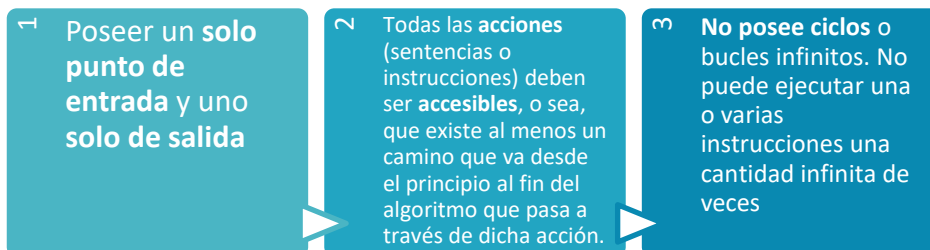
Se considera un primer borrador, igual que el diagrama de flujo, dado que el pseudocódigo tiene que traducirse posteriormente a un lenguaje de programación. Cabe señalar, que el pseudocódigo **no puede ser ejecutado** por una computadora.



Pero ¿qué es la Programación Estructurada?

La **programación estructurada** es un conjunto de técnicas y métodos para diseñar y escribir programas utilizando un método científico y no solamente el método prueba error. Se basa en dos cuestiones fundamentales:

1) Las **características de un algoritmo**



2) **Teorema de la Programación Estructurada**, que establece que todo Algoritmo estructurado puede ser escrito utilizando solamente tres tipos de estructuras de control de secuencia



Las **ventajas** de utilizar un Pseudocódigo en lugar de un Diagrama de Flujo son:

- Permite **representar en forma fácil** operaciones repetitivas complejas
- Es muy fácil **transformar una solución** de Pseudocódigo a un programa en algún lenguaje de programación.
- Si se siguen las reglas **se puede observar** claramente **los niveles** que tiene cada operación.

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

ELEMENTOS BÁSICOS DE LOS ALGORITMOS

A partir de determinado problema, vamos a iniciar su solución algorítmica. En primer lugar planteando una alternativa usando **Diagrama de Flujo**, luego expresando ese diseño en **pseudocódigo**.

Tipos de Datos

En Pseudocódigo vamos a utilizar los siguientes tipos de datos (Por ahora):



» NUMÉRICOS

Son aquéllos que representan una cantidad o valor determinado.

- **Enteros:** Es un conjunto finito de los números enteros. Los enteros son números completos, no tienen componentes fraccionarios o decimales y pueden ser negativos y positivos. Algunos ejemplos son: 10, 5, 0, -234, -3 ...
 - *Por ejemplo un dato que represente la edad de una persona, debe ser siempre del tipo Entero.*
- **Reales:** Consiste en un subconjunto de los números reales. Estos números siempre tienen un punto decimal y pueden ser positivos o negativos. Algunos ejemplos son: 0,345, 12.4, -231.005
 - *Por ejemplo un dato que represente el peso de una persona, debe ser siempre del tipo Real.*

» ALFANUMÉRICOS

Son los datos que representan información textual (palabras, frases, símbolos, etc). No representan valor alguno para efectos numéricos, o sea no podemos operar algebraicamente con ellos.

- *Por ejemplo el nombre de una persona, alguna frase cualquiera, etc*

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

» LÓGICOS

También se le denomina Booleano, es aquél dato que solo puede tomar uno de dos valores: Falso y verdadero.

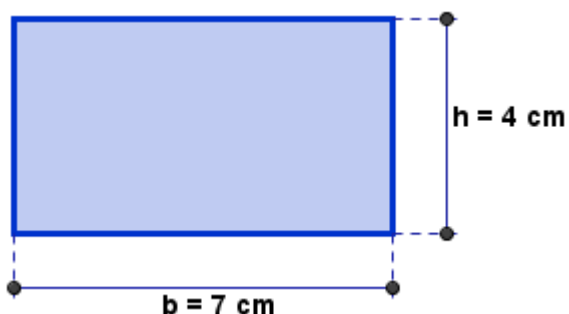
- Por ejemplo cuando se quiere determinar si un número es primo o no.

Estructuras de Datos simples

Las estructuras de datos Simples pueden clasificarse en:

- Variables
- Constantes

A la hora de elaborar un programa es necesario usar **datos**.



*Por ejemplo si necesitamos realizar el cálculo del área del rectángulo, para poder obtener el área del mismo, necesitamos almacenar en la memoria de la computadora el valor de la **base** y de la **altura**, para luego poder multiplicar sus valores.*

Recordemos que no es lo mismo grabar los datos en memoria que grabarlos en el disco duro. Cuando decimos grabar en memoria nos estaremos refiriendo a grabar esos datos en la RAM. Ahora bien, para grabar esos datos en la RAM podemos hacerlo utilizando dos elementos, llamados: **variables y constantes**.

Variables

En cualquier programa siempre necesitaremos hacer cálculos, usar información, procesarla y mostrarla. En la mayoría de los casos, necesitaremos un lugar temporal en el cual guardar parte de esa información, incluso toda.

Así las **variables**, son elementos de almacenamiento de datos.

Representan una dirección de memoria en donde se almacena un dato, que su contenido **puede variar** en el desarrollo del algoritmo.

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia



En una variable se puede almacenar distintos tipos de datos. De acuerdo al tipo de dato, definido para cada lenguaje de programación, será la cantidad de bytes que ocupa dicha variable en la memoria.

Unidades de Medidas de Almacenamiento

Medida	Simbología	Equivalencia	Equivalente en Bytes
byte	b	8 bits	1 byte
kilobyte	Kb	1024 bytes	1 024 bytes
megabyte	MB	1024 KB	1 048 576 bytes
gigabyte	GB	1024 MB	1 073 741 824 bytes
terabyte	TB	1024 GB	1 099 511 627 776 bytes
Petabyte	PB	1024 TB	1 125 899 906 842 624 bytes
Exabyte	EB	1024 PB	1 152 921 504 606 846 976 bytes
Zetabyte	ZB	1024 EB	1 180 591 620 717 411 303 424 bytes
Yottabyte	YB	1024 ZB	1 208 925 819 614 629 174 706 176 bytes
Brontobyte	BB	1024 YB	1 237 940 039 285 380 274 899 124 224 bytes
Geopbyte	GB	1024 BB	1 267 650 600 228 229 401 496 703 205 376 bytes

Lo más importante de la definición de las variables y la elección del tipo de datos asociados es el **significado de la variable**, o su semántica: ya que en base al tipo de datos seleccionado serán las operaciones que podemos realizar con esa variable, por ejemplo: si tenemos la variable edad deberíamos seleccionar un tipo de datos como entero ya que las operaciones relacionadas serán de comparación, sumas o restas y no es necesario tener una profundidad de decimales.

nombre_variable: tipo_de_dato

ELEMENTOS PARA LA DEFINICIÓN DE VARIABLES

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Constantes

Una **constante**, al igual que una variable, representa una zona de memoria en la cual se almacena un dato.

Sin embargo su contenido **no puede modificarse** durante la ejecución del algoritmo.



¿Cuándo usar constantes?

Se recomienda usar un dato constante, cuando voy a utilizar un valor definido y estático varias veces dentro de un programa.

Recuerda que ese dato ocupa espacio en memoria, por lo que no debemos definir constantes por doquier sin analizar realmente la necesidad de hacerlo

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Operadores y Operandos

Los programas de computadoras se apoyan esencialmente en la realización de numerosas operaciones aritméticas y matemáticas de diferente complejidad.

Operadores

Un **operador** es el símbolo que determina el tipo de operación o relación que habrá de establecerse entre los operandos para alcanzar un resultado.

Los operadores nos permiten manipular datos, sean variables, constantes, otras expresiones, objetos, atributos de objetos, entre otros, de manera que podamos:

- a) transformarlos,
- b) usarlos en decisiones para controlar el flujo de ejecución de un programa
- c) formar valores para asignarlos a otros datos

El tipo de datos involucrado en una expresión se relaciona con los operadores utilizados.

Te preguntará ¿Qué es una expresión?



3x2

Por ejemplo, en esta **expresión**, el símbolo **X** es el operador de MULTIPLICACIÓN, y los números 3 y 2 se llaman operandos

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

En síntesis, una **expresión** es una secuencia de operaciones y operandos que especifica un cálculo. Existen varios tipos de operadores:

Operadores de Asignación

Es el operador más simple que existe, se utiliza para **asignar un valor a una variable**. El signo que representa la asignación es el **:=** y este operador indica que el valor a la derecha del **:=** será asignado a lo que está a la izquierda del mismo.

Ejemplo en pseudocódigo:

```
Edad := 29
precio := 25.45
```



La acción de asignar es destructiva, ya que el valor que tuviera la variable antes de la asignación se pierde y se reemplaza por el nuevo valor.

Operadores Aritméticos

Son **operadores binarios** (requieren siempre dos operandos) que realizan las **operaciones aritméticas** habituales.

Utilizan operandos aritméticos y proporcionan resultados matemáticos.

Operador	Descripción
+	Suma
-	Resta
*	Multiplicación
/	División real
MOD	Resto de la División entera
DIV	Cociente de la División entera

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Operadores Relacionales

Los operadores relacionales sirven para **realizar comparaciones** de igualdad, desigualdad y relación de menor o mayor. Estos operadores sirven para expresar las condiciones en los algoritmos. Proporcionan resultados lógicos.

Operador	Significado
=	Igual a
<>	No igual a
>	Mayor que
>=	Mayor o igual que
<	Menor que
<=	Menor o igual que

El formato general para las comparaciones es:

expresión1 operador de relación expresión2

El resultado de la operación será Verdadero o Falso.

Así por ejemplo, si A=4 y B=3, entonces:

A > B Es Verdadero

(A-2) < (B-4) Es Falso

Los operadores de relación se pueden aplicar a varios tipos de datos: enteros, real, alfanumérico...

'A' < 'K' => Verdadero

'A' > 'a' => Falso

'MARIA' < 'JUAN' => Falso (se considera la primera letra)



Las comparaciones entre datos alfanuméricos se realizan utilizando código ASCII

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Operadores Lógicos

Son aquéllos que permiten la **combinación de condiciones** para formar **una sola expresión lógica**. Utilizan operandos lógicos y proporcionan resultados lógicos también.

Operador	Descripción	Uso
AND, ∧, Y	Operador lógico AND	a Y b
OR, ∨, O	Operador lógico OR	a O b

Para una mayor claridad, te dejamos aquí la tabla de valores de una expresión lógica:

A Y B		
A	B	RESULTADO
V	V	V
V	F	F
F	V	F
F	F	F

A O B		
A	B	RESULTADO
V	V	V
V	F	V
F	V	V
F	F	F

Debe notarse que en ciertos casos el segundo operando no se evalúa porque no es necesario (si ambos tienen que ser true y el primero es false ya se sabe que la condición de que ambos sean true no se va a cumplir).

Prioridad de los Operadores

Los operadores lógicos y matemáticos tienen un **orden de prioridad** o **precedencia**. Este es un esquema general que indica el orden en que deben evaluarse en la mayoría de los lenguajes de programación:

Prioridad de los operadores aritméticos, relacionales, lógicos y de cadena (de mayor a menor) en pseudocódigo:	
+ - no	Signo más, signo menos y negación
**	Potencia
* / div mod	Multipliación, división real, división entera y módulo
+ -	Suma y resta
+	Concatenación
< <= > >=	Menor que, menor o igual que, mayor que, mayor o igual que
= <>	Igual que y distinto que
Y	Conjunción
O	Disyunción

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

Prueba de Escritorio

Se denomina **prueba de escritorio** a la comprobación que se hace de un algoritmo para saber si está bien realizado. Esta prueba consiste en **tomar datos específicos como entrada** y seguir la secuencia indicada en el algoritmo hasta obtener un resultado, **el análisis de estos resultados** indicará si el **algoritmo está correcto** o si por el contrario **hay necesidad de corregirlo** o hacerle ajustes.



La herramienta fundamental que nos permitirá conocer cómo se comporta nuestro algoritmo es la prueba de escritorio y es una muy buena costumbre utilizarla.

¿Por qué debo usar la prueba de escritorio?

Las pruebas de escritorio son vitales en la formación de cualquier programador y son muchas las razones para afirmar esto, en este curso solo se mencionarán las 3 principales:

1. La primera razón es que las pruebas de escritorio son una estrategia para buscar errores en un programa.
2. La segunda razón, es que con una prueba de escritorio se puede entender cómo funciona un algoritmo.
3. Y finalmente, la razón más importante; las pruebas de escritorio desarrollan habilidades cognitivas que son esenciales para toda persona que programe.

Pasos para elaborar una Prueba de Escritorio

Este trabajo se realiza en base a una tabla cuyos encabezados son las **variables** que se usan en el algoritmo y debajo de cada una de ellas se van colocando los valores que van tomando, paso a paso y siguiendo el flujo indicado por el algoritmo, hasta llegar al final. Cada fila de la tabla representará el estado de las variables en un instante determinado.

Por ejemplo:

1. $a := 2$;
2. $b := 3$;
3. $\text{suma} := a + b$;
4. $\text{suma} := \text{suma} + 1$;

ESTADO	a	b	suma
$E_0 = E_{\text{inicial}}$	¿?	¿?	¿?
E_1	2	¿?	¿?
E_2	2	3	¿?
E_3	2	3	5
$E_4 = E_{\text{final}}$	2	3	6

ALGORITMOS Y ESTRUCTURAS DE DATOS

UNIDAD I: Introducción a la Algoritmia

La prueba consistirá en 2 etapas:

- La primera, en probar inicialmente que el programa funcione correctamente, para lo que se elegirá algunos datos fáciles de probar, cosa que siempre es posible.
- La segunda, si se prueba que ya funciona, se buscarán otros datos (si los hay) que hagan que falle el algoritmo, en cuyo caso se habrán de detectar otros errores.
 - Si el algoritmo no falla, podemos concluir que el programa está terminado y revisado, por lo tanto, correcto.